第31课 栈与队列（STL）

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：认识栈的基本操作（入门）

题目：依次往栈中放入 1、2、3、4、5，然后一个一个取出来并打印，观察输出顺序。

讲解思路：

> 我们往盘子堆上依次放 1 号、2 号、3 号、4 号、5 号盘子。现在从顶上一个个拿，最先拿到的是 5 号——因为它是最后放上去的！

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s;

    // 依次放入 1, 2, 3, 4, 5
    for (int i = 1; i <= 5; i++) {
        s.push(i);
        cout << "放入: " << i << endl;
    }

    cout << "栈里一共有 " << s.size() << " 个元素" << endl;
    cout << "---开始取出---" << endl;

    // 一个一个取出来
    while (!s.empty()) {
        cout << "取出: " << s.top() << endl;  // 先看
        s.pop();                              // 再删
    }

    cout << "栈空了吗? " << (s.empty() ? "是" : "否") << endl;

    return 0;
}
```

预期输出：

```
放入： 1
放入： 2
放入： 3
放入： 4
放入： 5
栈里一共有 5 个元素
---开始取出---
取出： 5
取出： 4
取出： 3
取出： 2
取出： 1
栈空了吗？是
```

要点强调：放入顺序 15，取出顺序 51，完美体现"后进先出"！

例题 2：认识队列的基本操作（入门）

题目：模拟 5 位同学排队：依次编号 1~5 进入队列，然后一个一个离开，观察输出顺序。

讲解思路：

> 食堂开门了！1 号同学先到，排在最前面；2 号第二个到……5 号最后到。现在开始打饭，谁先打？当然是 1 号！

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    // 5 位同学依次排队
    for (int i = 1; i <= 5; i++) {
        q.push(i);
        cout << i << " 号同学排到队尾" << endl;
    }

    cout << "队伍长度: " << q.size() << endl;
    cout << "队首: " << q.front() << " 号" << endl;
    cout << "队尾: " << q.back() << " 号" << endl;
    cout << "---开始打饭---" << endl;

    // 一个一个离开
    while (!q.empty()) {
        cout << q.front() << " 号同学打完饭走了" << endl;
        q.pop();
    }

    cout << "队伍空了吗? " << (q.empty() ? "是" : "否") << endl;

    return 0;
}
```

预期输出：

```
1 号同学排到队尾
2 号同学排到队尾
3 号同学排到队尾
4 号同学排到队尾
5 号同学排到队尾
队伍长度： 5
队首： 1 号
队尾： 5 号
---开始打饭---
1 号同学打完饭走了
2 号同学打完饭走了
3 号同学打完饭走了
4 号同学打完饭走了
5 号同学打完饭走了
队伍空了吗？是
```

要点强调：进入顺序 15，离开顺序也是 15，完美体现"先进先出"！与栈正好相反。

例题 3：括号匹配（栈的应用，进阶）

题目：输入一个只包含 `(` 和 `)` 的字符串，判断括号是否匹配。

讲解思路：

> 想象你有一个盒子（栈）。从左到右读字符串——
> - 看到 `(`：放一个小球进盒子，代表"这里有个左括号在等配对"。
> - 看到 `)`：去盒子里拿一个小球出来配对。如果盒子是空的，说明右括号多了！
> - 读完后看盒子：空的说明全部配对完毕，还有小球说明左括号多了。

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    string str;
    cout << "请输入括号字符串: ";
    cin >> str;

    stack<char> s;
    bool ok = true;  // 记录是否匹配

    for (int i = 0; i < str.size(); i++) {
        if (str[i] == '(') {
            s.push('(');  // 左括号入栈
        } else if (str[i] == ')') {
            if (s.empty()) {
                // 栈空了还遇到右括号，说明右括号多了
                ok = false;
                break;
            }
            s.pop();  // 配对成功，弹出一个左括号
        }
    }

    // 如果到最后栈不为空，说明左括号多了
    if (!s.empty()) {
        ok = false;
    }

    if (ok) {
        cout << "括号匹配正确! " << endl;
    } else {
        cout << "括号匹配错误! " << endl;
    }

    return 0;
}
```

测试用例：

输入	输出	说明
`(()()()`	括号匹配正确！	每个左括号都找到了配对
`(()`	括号匹配错误！	多了一个左括号
`())`	括号匹配错误！	多了一个右括号
`()()()`	括号匹配正确！	三对括号，完美配对

逐步模拟（以 `(()()()` 为例）：

读到字符	操作	栈内容（底顶）
`(`	入栈	`(`
`(`	入栈	`( (`
`)`	配对，弹出	`(`
`)`	配对，弹出	空
`(`	入栈	`(`
`)`	配对，弹出	空

最终栈为空 匹配正确

例题 4：模拟排队过程（队列的应用，进阶）

题目：有 n 个人排队买冰淇淋，编号 1~n。每个人买完需要 1 分钟。但如果某人的编号是 3 的倍数，他会买完后再排到队尾重新排一次（太好吃了想再买一个）。问最终所有人的离开顺序。

讲解思路：

> 这就像真实排队！用队列模拟：每次让队首的人买完离开。但如果他的编号是 3 的倍数（比如 3 号、6 号），他离开后又跑去队尾重新排队。不过第二次买完就真的走了，不会再排第三次。

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    int n;
    cout << "请输入排队人数: ";
    cin >> n;

    queue<int> q;            // 排队队列，存编号
    int bought[101] = {0};  // 记录每个人已经买了几次

    // 所有人排队
    for (int i = 1; i <= n; i++) {
        q.push(i);
    }

    cout << "离开顺序: ";
    while (!q.empty()) {
        int person = q.front();  // 看看队首是谁
        q.pop();                 // 离开队首

        bought[person]++;  // 买了一次

        if (person % 3 == 0 && bought[person] == 1) {
            // 编号是3的倍数，且是第一次买，重新排队
            q.push(person);
        } else {
            // 真正离开
            cout << person << " ";
        }
    }
    cout << endl;

    return 0;
}
```

测试（n = 6）：

初始队列：`1 2 3 4 5 6`

步骤	队首离开	操作	队列状态	输出
1	1	离开	`2 3 4 5 6`	1
2	2	离开	`3 4 5 6`	2
3	3	3是3的倍数，第1次重新排队	`4 5 6 3`	—
4	4	离开	`5 6 3`	4
5	5	离开	`6 3`	5
6	6	6是3的倍数，第1次重新排队	`3 6`	—
7	3	第2次，离开	`6`	3
8	6	第2次，离开	空	6

最终输出： `1 2 4 5 3 6`

附：例题源代码与运行结果

例题 1：Basic Stack Operations

```
// Push 1~5 onto a stack, then pop them one by one to observe LIFO order.
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s;

    // Push 1, 2, 3, 4, 5 onto the stack
    for (int i = 1; i <= 5; i++) {
        s.push(i);
        cout << "Pushed: " << i << endl;
    }

    cout << "Stack size: " << s.size() << endl;
    cout << "--- Start popping ---" << endl;

    // Pop elements one by one
    while (!s.empty()) {
        cout << "Popped: " << s.top() << endl;  // Peek at top
        s.pop();                                // Remove top
    }

    cout << "Is stack empty? " << (s.empty() ? "Yes" : "No") << endl;

    return 0;
}
```

Expected Output

```
Pushed: 1
Pushed: 2
Pushed: 3
Pushed: 4
Pushed: 5
Stack size: 5
--- Start popping ---
Popped: 5
Popped: 4
Popped: 3
Popped: 2
Popped: 1
Is stack empty? Yes
```

例题 2：Basic Queue Operations

```
// Simulate 5 people lining up (numbered 1~5), then leaving one by one.
// Demonstrates FIFO order.

/*
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    // 5 people join the queue
    for (int i = 1; i <= 5; i++) {
        q.push(i);
        cout << "Person " << i << " joined the queue" << endl;
    }

    cout << "Queue size: " << q.size() << endl;
    cout << "Front: Person " << q.front() << endl;
    cout << "Back: Person " << q.back() << endl;
    cout << "--- Start serving ---" << endl;

    // People leave one by one from the front
    while (!q.empty()) {
        cout << "Person " << q.front() << " left the queue" << endl;
        q.pop();
    }

    cout << "Is queue empty? " << (q.empty() ? "Yes" : "No") << endl;

    return 0;
}
*/
```

Expected Output

```
Person 1 joined the queue
Person 2 joined the queue
Person 3 joined the queue
Person 4 joined the queue
Person 5 joined the queue
Queue size: 5
Front: Person 1
Back: Person 5
--- Start serving ---
Person 1 left the queue
Person 2 left the queue
Person 3 left the queue
Person 4 left the queue
Person 5 left the queue
Is queue empty? Yes
```

例题 3：Parenthesis Matching (Stack Application)

```
// Check if a string of '(' and ')' has valid matching.

/*
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    string str;
    cout << "Enter a parenthesis string: ";
    cin >> str;

    stack<char> s;
    bool ok = true;

    for (int i = 0; i < str.size(); i++) {
        if (str[i] == '(') {
            s.push('(');  // Push left parenthesis
        } else if (str[i] == ')') {
            if (s.empty()) {
                // Extra right parenthesis
                ok = false;
                break;
            }
            s.pop();  // Match found, pop the left parenthesis
        }
    }

    // If stack is not empty, there are extra left parentheses
    if (!s.empty()) {
        ok = false;
    }

    if (ok) {
        cout << "Parentheses match correctly!" << endl;
    } else {
        cout << "Parentheses do NOT match!" << endl;
    }

    return 0;
}
*/
```

Sample Input

```
(())()
```

Expected Output

```
Enter a parenthesis string: (())()
Parentheses match correctly!
```

例题 4：Queue Simulation — Ice Cream Line

```
// n people line up (numbered 1~n). Each person takes 1 minute.
// If a person's number is a multiple of 3, they rejoin the queue once.
// Output the leaving order.

/*
#include <iostream>
#include <queue>
using namespace std;

int main() {
    int n;
    cout << "Enter number of people: ";
    cin >> n;

    queue<int> q;
    int bought[101] = {0};  // Track how many times each person has bought

    // Everyone joins the queue
    for (int i = 1; i <= n; i++) {
        q.push(i);
    }

    cout << "Leaving order: ";
    while (!q.empty()) {
        int person = q.front();
        q.pop();

        bought[person]++;

        if (person % 3 == 0 && bought[person] == 1) {
            // Multiple of 3 and first purchase — rejoin the queue
            q.push(person);
        } else {
            // Actually leave
            cout << person << " ";
        }
    }
    cout << endl;

    return 0;
}
*/
```

Sample Input

```
6
```

Expected Output

```
Enter number of people: 6
Leaving order: 1 2 4 5 3 6
```